ROUND 1 · ONLINE PROPOSAL SUBMISSION

PayShield

Constraint-Aware Fraud Intelligence for Payment Decisions

"Detect fraud without turning every suspicious transaction into customer friction."



PROBLEM STATEMENT
PS-02 · SYN26-CS02 · FinTech [ANIRBAN SHIT]

TEAM
DEV_X

INSTITUTION
Jalpaiguri Govt Engineering College (JGEC)

TEAM MEMBERS
ANIRBAN SHIT · SANTANU MAITY · TITAS MAL · AGNIVA GHOSH

What This Submission Covers

Constraint-Aware Fraud Intelligence for Payment Decisions

01 Problem Statement Overview

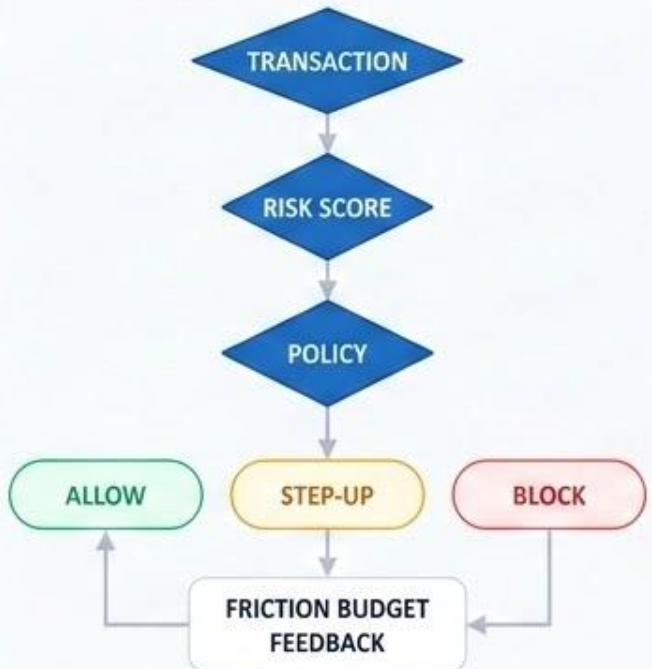
02 Problem & Domain Grounding

03 Existing Approaches & Gap

04 Proposed Solution

05 Architecture — Perception Layer

06 Architecture — Decision / Policy Layer



07 Feasibility & Free-Compute Stack

08 Feasibility & Viability Analysis

09 Trade-off Formulation

10 Team, Roles & Roadmap

SECTION 01

Problem Statement Overview

PROBLEM STATEMENT NO. & TITLE

PS-02 · SYN26-CS02 · PayShield

DOMAIN / TRACK

FinTech / Applied Machine Learning

TEAM LEVEL

B.Tech Baseline

RECOMMENDED DATASET

IEEE-CIS Fraud Detection / ~150 MB / Kaggle

ONE-PARAGRAPH RESTATEMENT

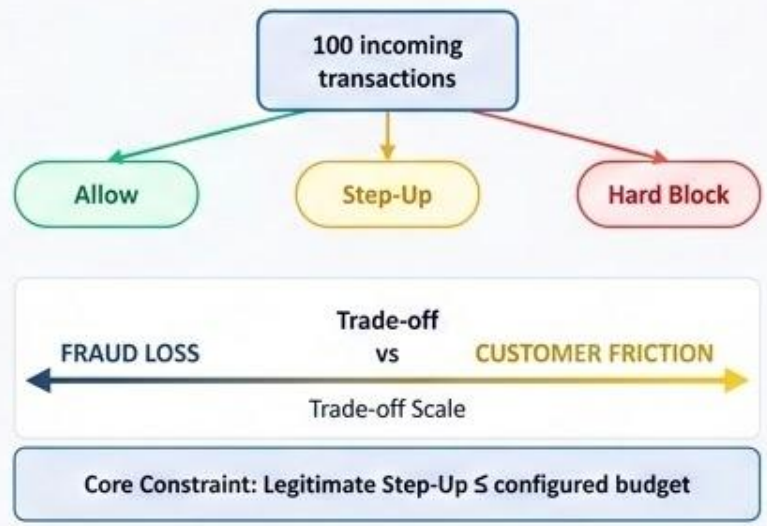
Payment platforms must distinguish fraudulent transactions from legitimate customer activity in real time.

A binary allow/deny system creates a difficult trade-off: aggressive blocking can prevent fraud but can also reject genuine customers, while permissive policies can allow costly fraud.

PayShield treats fraud prevention as a three-action decision problem: Allow, Step-Up Verification, or Hard Block.

A machine-learning model produces a calibrated transaction risk score, while a separate policy layer converts that score into an action under an explicit customer-friction budget.

The key requirement is that the system can change the friction budget live and immediately recalibrate its Step-Up threshold without retraining or restarting.



Problem & Domain Grounding

THE REAL-WORLD BOTTLENECK

Fraud detection is not only a classification problem. The operational system must decide how much customer friction to introduce for a given level of fraud risk.

MORE BLOCKING ↓
- More fraud intercepted
- More legitimate customers affected



LESS FRICTION ↓
- Better customer experience
- More fraud exposure

WHY IT MATTERS — IMPACT IF UNSOLVED

- Fraud creates direct financial loss and investigation/recovery cost.
- False positives can interrupt legitimate purchases and increase checkout abandonment.
- A fixed threshold cannot directly express an operational friction budget.
- Fraud patterns and transaction distributions can shift over time.

OPERATIONAL FAILURE MODES



Static threshold
A fixed Step-Up threshold cannot respond to a changed friction budget.



Binary decisioning
Allow/deny logic has no controlled middle action for uncertain transactions.



Uncalibrated scores
Raw model scores are **not** reliable probabilities for policy decisions.



Budget violation
A policy may catch more fraud while **exceeding** the permitted legitimate-customer Step-Up rate.

SECTION 03

Existing Approaches & Gap

	APPROACH	HOW IT WORKS	ILLUSTRATIVE EXAMPLE	KEY LIMITATION
	1. RULE-BASED FRAUD RULES	Expert-defined rules using transaction, velocity, device, geography or account signals.	**Illustrative rule:** IF 5+ transactions occur within 10 min AND device is new → trigger review	Rules are explicit and interpretable, but require maintenance and can become brittle as fraud patterns evolve.
	2. BINARY ML CLASSIFIER	Train a classifier to estimate whether a transaction is fraudulent.	**Illustrative threshold:** Model outputs: P(fraud) = 0.87 Fixed decision: ≥ 0.80 → Fraud, < 0.80 → Legiti	The model predicts risk/class membership, but the final operating point does not inherently encode a customer-friction budget.
	3. STATIC RISK THRESHOLD	Convert a fraud-risk score into an action using a fixed operating threshold.	Risk score = 0.73, Threshold = 0.70 → STEP-UP Budget changes: 2% → 1% Fixed threshold → unchanged	A fixed threshold does not automatically adapt when the desired legitimate-customer friction budget changes.
	4. COST-SENSITIVE CLASSIFICATION	Use asymmetric costs so missed fraud and false alarms have different penalties.	False negative: ₹10,000 fraud loss, False positive: ₹100 customer-friction cost. Optimize expected decision cost	A fixed cost configuration does not directly guarantee that legitimate Step-Up actions stay within a live operational budget.

Examples are illustrative; thresholds and costs are deployment-specific.

THE GAP — PAYSHIELD TARGETS

Fraud scoring answers: 'How risky is this transaction?' **Policy answers:** 'What action should we take under today's friction constraint?'

CONVENTIONAL
↓
Risk Score → Fixed Threshold
↓
Action

versus

PAYSHIELD
↓
Risk Score → Dynamic Policy ← Friction Budget B
↓
ALLOW | STEP-UP | BLOCK

SAME MODEL + NEW FRICTION BUDGET → NEW STEP-UP THRESHOLD
Example:
2% → $T_{step\text{-}up}(2\%)$, 1% → $T_{step\text{-}up}(1\%)$
NO MODEL RETRAINING, NO SYSTEM RESTART

[1] Al-Hashedi & Magalingam, 'Financial fraud detection applying data mining techniques: A comprehensive review from 2009 to 2019', Computer Science Review, 2021. [2] 'How Artificial Intelligence and machine learning research impacts payment card fraud detection: A survey and industry benchmark', Engineering Applications of Artificial Intelligence, 2018, DOI: 10.1016/j.engappai.2018.07.008. [3] 'Financial Fraud

SECTION 04

Proposed Solution

PayShield separates fraud intelligence from transaction policy: a calibrated ML scorer estimates risk, while a dynamic policy engine converts risk into Allow, Step-Up, or Block under a live friction budget.

1 Core Idea

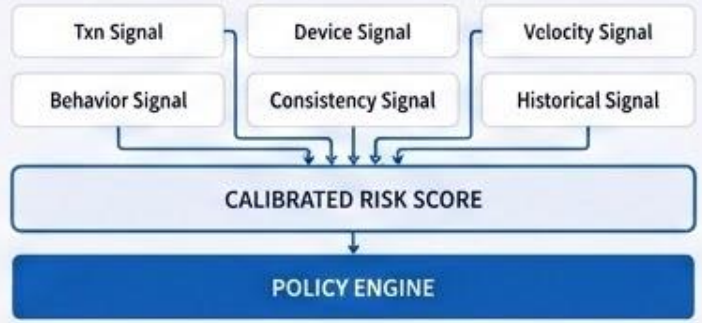
"Score first. Decide second."



Key Takeaway: The ML model remains stable while the policy thresholds adapt to the current operational constraint.

2 Cooperative Mechanism

"Risk Echo"

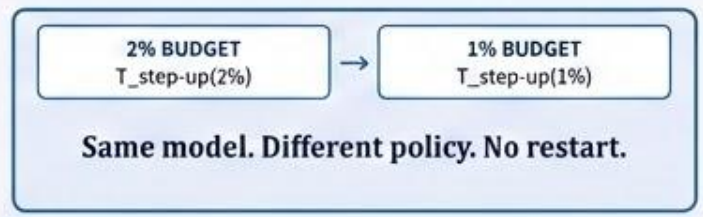


Explain: "The 'echo' is the propagation of independent transaction signals into a shared risk representation and then into the policy layer."

3 What's New

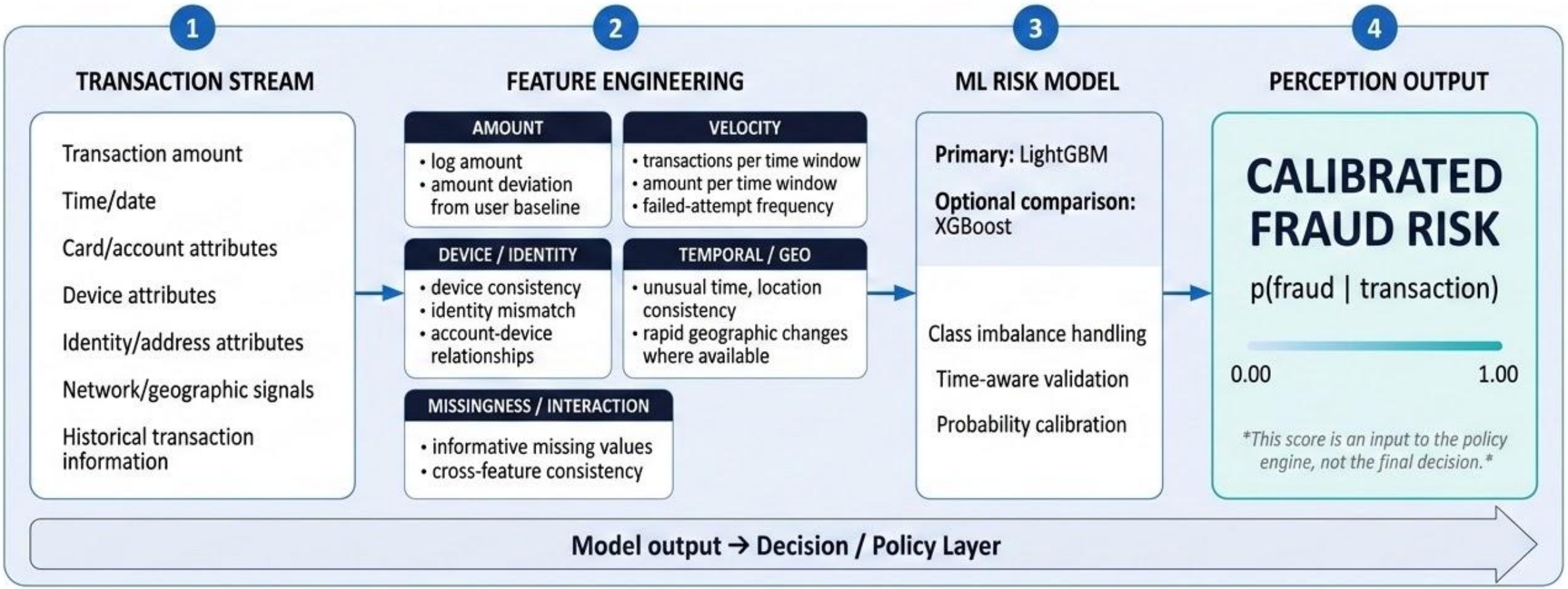
Architectural Innovation

- **Three-way action space** instead of binary allow/deny.
- **Explicit friction-budget** constraint over legitimate transactions.
- **Threshold recalibration** when the budget changes — without retraining the ML model.



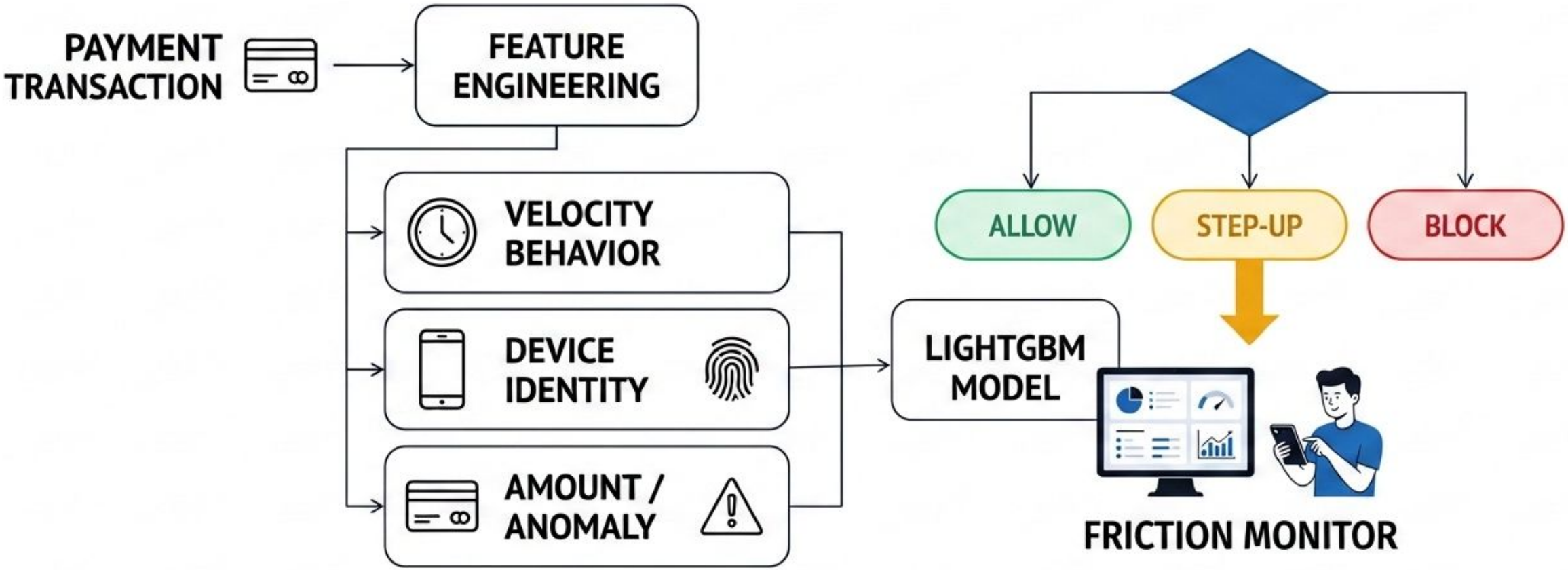
Same model. Different policy. No restart.

System Architecture — Perception Layer

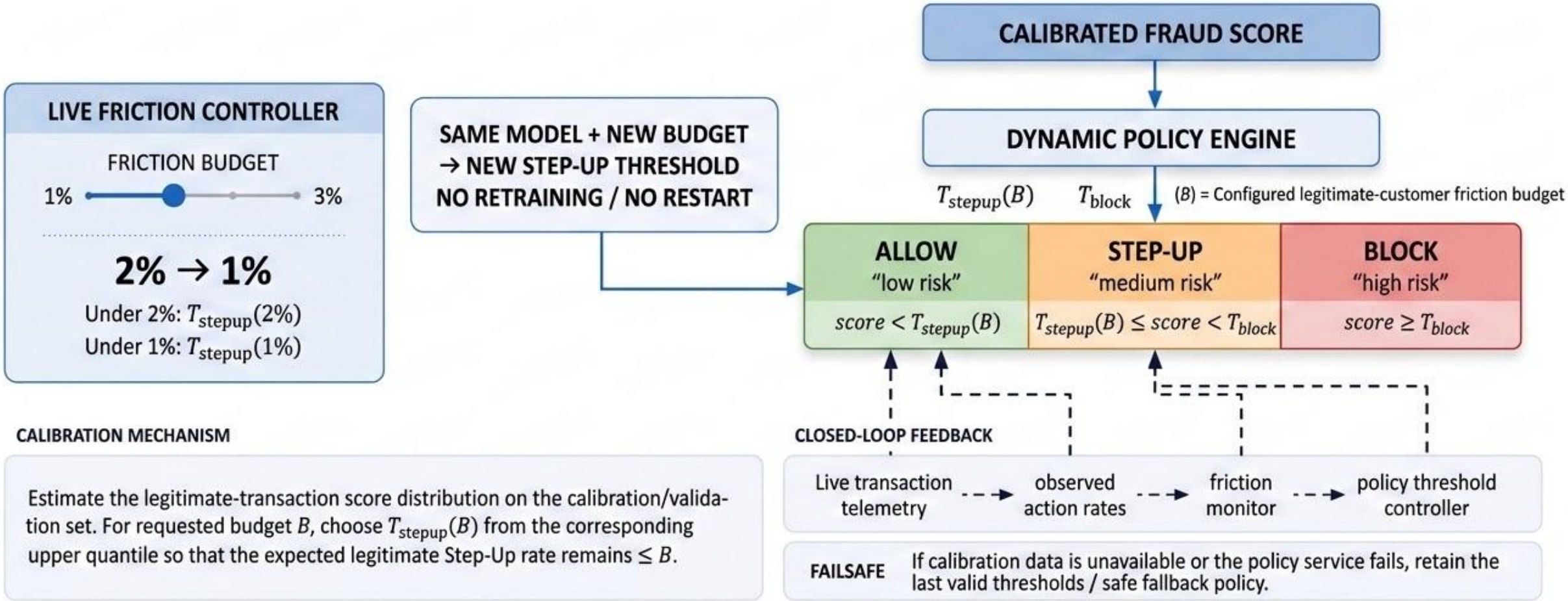


Architecture — Perception Layer

Constraint-Aware Fraud Intelligence for Payment Decisions



System Architecture — Decision / Policy Layer



CALIBRATION MECHANISM

Estimate the legitimate-transaction score distribution on the calibration/validation set. For requested budget B, choose $T_{stepup}(B)$ from the corresponding upper quantile so that the expected legitimate Step-Up rate remains $\leq B$.

FAILSAFE If calibration data is unavailable or the policy service fails, retain the last valid thresholds / safe fallback policy.

Feasibility & Free-Compute Stack

An end-to-end fraud decision platform built with open-source and free development tools

USER / TRANSACTION →

01 — WEB / USER LAYER

Streamlit

Technologies: Streamlit

Functions:
- Transaction simulator
- Fraud-risk dashboard
- Friction-budget control
- Live decision visualization

→

02 — BACKEND / API

FastAPI

Technologies: FastAPI

Functions:
- REST API
- JSON transaction input
- Request validation
- Model inference endpoint

→

03 — DATA / FEATURE ENGINE

Technologies: pandas, NumPy, scikit-learn

Feature families:
- Amount anomalies
- Transaction velocity
- Device / identity
- Temporal / geographic
- Historical behavior
- Missingness / interactions

→

04 — ML FRAUD ENGINE

LightGBM

Primary model: LightGBM

Optional comparison: XGBoost

Calibration: scikit-learn calibration

Output: CALIBRATED FRAUD RISK SCORE

→

05 — DYNAMIC POLICY ENGINE

Friction Budget B
Dynamic $T_{step\text{-}up}(B)$
Fixed T_{block}

Adjusts action thresholds to satisfy legitimate-customer friction constraints.

SAME MODEL + NEW BUDGET → NEW STEP-UP THRESHOLD

2% → $T_{step\text{-}up}(2\%)$
1% → $T_{step\text{-}up}(1\%)$

NO MODEL RETRAINING NO SYSTEM RESTART

→

06 — DECISION

ALLOW low risk

STEP-UP medium risk

BLOCK high risk

→

07 — MONITORING / ANALYTICS

plotly

Technologies: Plotly

Analytics:
- Fraud recall
- Legitimate Step-Up %
- Decision breakdown
- Fraud dollars prevented

DEPLOYMENT & STACK

Deployment: Streamlit Community Cloud (Free tier)
Stack: Free-Compute

Feasibility & Viability Analysis

TECHNICAL

Feasible with tabular ML, calibrated probabilities and lightweight policy logic.

OPERATIONAL

Policy thresholds are independent from model training, enabling live friction-budget changes.

ECONOMIC

Designed around public data and free/open-source tooling; no paid API is required for the core demo.

SCALABILITY

Model scoring and policy evaluation are separable services, allowing independent scaling.

RISK	LIKELIHOOD	MITIGATION
Data leakage / temporal leakage	Medium	Time-aware validation and careful construction of historical features.
Model probability miscalibration	Medium	Calibration set + reliability analysis + threshold policy separated from raw model scores.
Judge changes budget during demo	High	Precomputed legitimate score distribution / quantile lookup and instant threshold recalculation.
Stream or dashboard failure	Low/Medium	Cached model + last-valid-policy fallback.

DESIGN PRINCIPLE:
"Keep model inference stable; make operational policy dynamically configurable."

MINI STRESS-TEST

2% → 1% (Budget changes) → Threshold recalculates → Step-Up policy updates → No model retraining

Trade-off Formulation

OBJECTIVE FUNCTION

maximize Expected Fraud Loss Prevented
$-\lambda_1 \times$ Legitimate Step-Up Cost $-\lambda_2 \times$ Legitimate Block Cost

[Equivalently described as:]

maximize $\sum$ fraud_transaction [caught_fraud_value]
$- C_{\text{stepup}} - C_{\text{false_block}}$

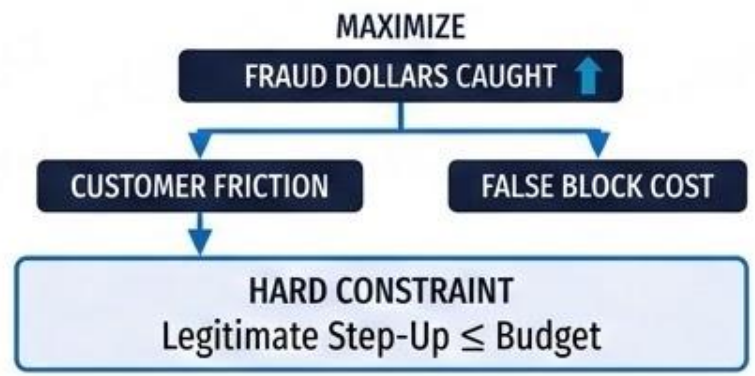
CONSTRAINTS

- Legitimate Step-Up Rate $\leq B$
- $B \in [1\%, 3\%]$ for the demo control
- Policy thresholds remain consistent with calibrated score ordering
- Inference remains within the demo latency budget

DOMAIN METRICS & TARGETS

Note: These are evaluation targets and metrics - are evaluation targets and metrics, not claimed results.

METRIC	TARGET
Fraud recall at fixed friction	Report at 1%, 2%, 3% budgets
Legitimate Step-Up rate	≤ configured budget
Fraud dollars caught	Maximize
False-positive / legitimate block rate	Minimize
Policy recalibration	Immediate / no restart
Decision breakdown	Allow + Step-Up + Block



Team, Roles & Roadmap

TEAM NAME: DEV_X
INSTITUTION: Jalpaiguri Government Engineering College (JGEC)

TEAM

AGNIVA GHOSH
ML / Fraud Detection Lead

RESPONSIBILITIES
- IEEE-CIS data preparation
- Feature engineering
- LightGBM fraud model
- Model evaluation
- Probability calibration

TITAS MAL
Backend / API Engineer

RESPONSIBILITIES
- FastAPI service
- Transaction inference API
- Request validation
- Model-serving integration
- Backend reliability

SANTANU MAITY
Frontend / Dashboard Engineer

RESPONSIBILITIES
- Streamlit interface
- Transaction simulator
- Risk visualization
- Decision dashboard
- Live friction-budget controls

ANIRBAN SHIT
Policy / Evaluation & Integration

RESPONSIBILITIES
- Dynamic threshold policy
- Friction-budget recalibration
- Evaluation metrics
- Stress-test integration
- End-to-end system integration

ROADMAP TOWARD THE GRAND FINALE

01 — DATA & BASELINE
TASKS
- Load IEEE-CIS dataset
- Time-aware validation split
- Feature engineering
- Establish fraud baseline

DELIVERABLE
"Clean training / validation pipeline"

BUIL

02 — MODEL & CALIBRATION
TASKS
- Train LightGBM
- Compare XGBoost if useful
- Calibrate probabilities
- Evaluate fraud recall / false positives

DELIVERABLE
"Calibrated fraud-risk scorer"

BUIL

03 — SYSTEM INTEGRATION
TASKS
- FastAPI inference service
- Streamlit dashboard
- Dynamic policy engine
- ALLOW / STEP-UP / BLOCK

DELIVERABLE
"End-to-end PayShield prototype"

BUIL

04 — STRESS TEST & DEMO
TASKS
- Set friction budget = 2%
- Change budget → 1%
- Recalculate Step-Up threshold
- Verify no model retraining / restart
- Present live decision flow

DELIVERABLE
"Live friction-budget stress test"

THANK YOU

Questions & discussion welcome

TEAM CONTACT EMAIL	PROTOTYPE LINK	TEAM NAME
[anirbanshit77@gmail.com]	[https://payshield-bay.vercel.app/overview]	[**DEV_X**]

SYNAPSE 1.0 · IEEE SMC KGEC × IEEE Kolkata Section